



Carrera:
Programación y WebMaster

Docente:
Mario Alonso Segovia Gutiérrez

Materia:
Proyecto de una Plataforma Virtual

Alumno(s):
Castro Blanco Manuel David

Fecha:
02 de Julio 2026

Tabla de contenido

<i>Introducción</i>	<i>3</i>
<i>Análisis del caso</i>	<i>3</i>
<i>Importancia del control de versiones.....</i>	<i>4</i>
<i>Propuesta de organización del proyecto.....</i>	<i>5</i>
<i>Diagrama del flujo de trabajo.....</i>	<i>6</i>
<i>Respuestas de análisis</i>	<i>7</i>
<i>Conclusión</i>	<i>8</i>

Introducción

En el desarrollo de software es muy común que varias personas trabajen al mismo tiempo sobre un mismo proyecto. Cuando eso sucede sin una forma ordenada de administrar el código, los problemas no tardan en aparecer: archivos que se pisan unos a otros, cambios que se pierden y nadie sabe con certeza quién modificó qué. Por eso el control de versiones se ha vuelto una herramienta básica para cualquier equipo de programación.

En este trabajo analizo el caso del proyecto SmartLibrary, una plataforma web para administrar una biblioteca digital que estaba siendo desarrollada por cinco programadores. El equipo comenzó compartiendo el proyecto mediante una simple carpeta en la nube y, con el paso de las semanas, esa decisión les trajo varias complicaciones. A lo largo del documento explico cuáles fueron los errores que cometieron, qué es y para qué sirve un sistema de control de versiones, y propongo una forma de organizar el trabajo usando Git. También incluyo un diagrama del flujo que debería seguir un desarrollador y respondo a varias preguntas de análisis sobre la situación.

Análisis del caso

1. ¿Cuáles fueron los principales errores cometidos por el equipo durante el desarrollo?

El error de fondo fue no usar ninguna herramienta de control de versiones y confiar todo a una carpeta compartida en la nube. A partir de ahí se desprenden varios errores concretos: cada desarrollador reemplazaba la carpeta completa con su propia copia, lo que provocaba que el trabajo de los demás se borrara; no había forma de combinar los cambios de dos personas, así que cuando dos tocaban el mismo archivo uno terminaba sobrescribiendo al otro; no existía un historial que permitiera regresar a una versión anterior cuando algo salía mal; y tampoco había manera de saber quién había hecho cada modificación, por lo que era imposible rastrear de dónde venían los errores.

2. ¿Por qué trabajar únicamente compartiendo carpetas genera tantos problemas?

Porque una carpeta compartida no entiende de “versiones”: solo guarda el último archivo que se copió encima. No sabe combinar dos modificaciones distintas del mismo archivo, no lleva un registro de los cambios y no protege el trabajo anterior. Si dos personas editan al mismo tiempo, la que guarda al último gana y la otra simplemente pierde lo que hizo. Además, no hay autoría ni fechas, así que cuando aparece un error nadie puede saber en qué momento se introdujo ni quién lo hizo.

3. ¿Qué consecuencias podría tener esta forma de trabajo en proyectos grandes?

En un proyecto grande las consecuencias se multiplican. Se pierde mucho tiempo rehaciendo trabajo que ya se había hecho, la confianza dentro del equipo se acaba porque a nadie le gusta que le borren su avance, y la calidad del producto baja porque se acumulan errores que nadie logra ubicar. También se vuelve casi imposible coordinar entregas o regresar a una versión estable cuando una actualización rompe algo.

Importancia del control de versiones

¿Qué es un sistema de control de versiones?

Es una herramienta que registra y guarda cada cambio que se hace en los archivos de un proyecto a lo largo del tiempo. En lugar de tener una sola carpeta que se va sobrescribiendo, el sistema conserva un historial completo: sabe qué se cambió, cuándo se cambió y quién lo hizo. Gracias a eso es posible volver a cualquier versión anterior cuando se necesite y trabajar entre varias personas sin tocar el trabajo de los demás.

¿Cuál es su principal objetivo?

Su objetivo principal es administrar de forma ordenada los cambios en el código fuente y permitir que varias personas colaboren en el mismo proyecto sin perder información. Busca que nada del trabajo se borre por accidente, que siempre se pueda regresar a un estado estable y que la integración de los avances de cada quien sea controlada y no un choque de archivos.

¿Qué ventajas ofrece frente al manejo tradicional de archivos?

Ofrece varias ventajas claras. Guarda un historial completo, por lo que se puede deshacer un cambio o volver a una versión que sí funcionaba. Permite que varias personas trabajen a la vez y luego combina sus cambios en lugar de sobrescribirlos. Registra la autoría, así que se sabe quién hizo cada cosa. Hace posible separar el trabajo en ramas para no afectar la versión estable mientras se desarrolla algo nuevo. Y, en general, da orden y seguridad al proyecto, cosa que una carpeta compartida nunca podrá ofrecer.

Propuesta de organización del proyecto

Suponiendo que el equipo de SmartLibrary ahora usará Git, propongo organizar el trabajo de la siguiente manera, sin entrar en comandos sino en la forma de coordinarse:

¿Qué rama utilizarías para la versión estable del sistema?

Usaría la rama principal, normalmente llamada main, para mantener la versión estable del sistema. En esa rama solo debería estar el código que ya fue probado y que funciona correctamente, porque representa lo que se podría poner en producción en cualquier momento.

¿Qué rama utilizarías para integrar el trabajo del equipo?

Para juntar el trabajo de todos usaría una rama de integración, que se suele llamar develop. Ahí se reúnen las funciones que cada desarrollador va terminando, se prueban juntas y, una vez que todo está estable, se pasan a la rama principal. De esta forma main nunca recibe código a medio terminar.

¿Cada desarrollador debería trabajar directamente sobre la rama principal? Explica tu respuesta.

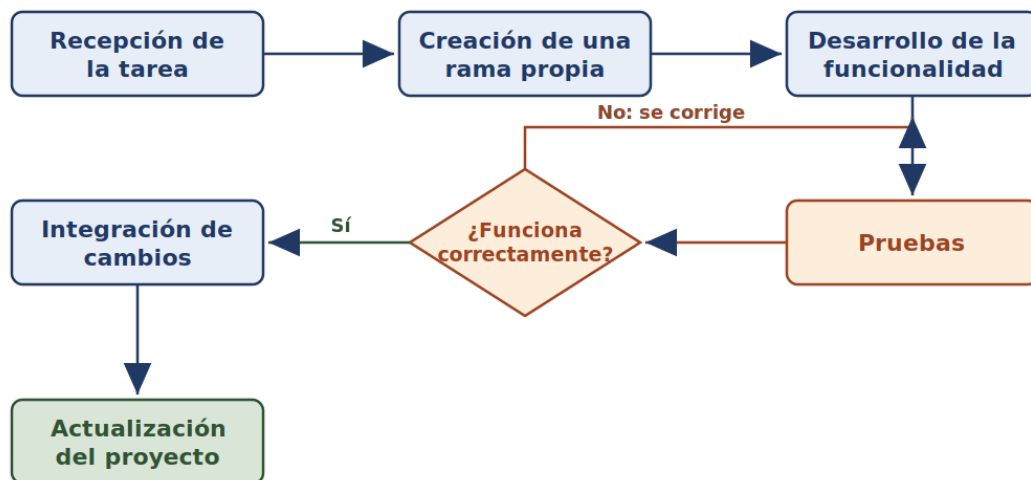
No. Cada desarrollador debería crear su propia rama para la tarea que le toca (por ejemplo, una rama para el módulo de autenticación, otra para el de usuarios, y así). Trabajar directamente sobre la rama principal es justo lo que causaba los problemas del inicio: todos chocando en el mismo lugar. Al usar ramas separadas, cada quien avanza sin estorbar a los demás y la versión estable se mantiene protegida hasta que el cambio esté listo y probado.

¿Cuándo deberían integrarse los cambios al proyecto?

Los cambios deberían integrarse cuando la funcionalidad esté terminada y haya pasado sus pruebas, nunca antes. Lo recomendable es revisar el código (por ejemplo, mediante una solicitud de integración o pull request) y, si todo está bien, unirlo primero a la rama de integración y luego a la principal. Así se evita meter errores al proyecto y se conserva siempre una versión funcional.

Diagrama del flujo de trabajo

El siguiente diagrama representa el proceso que seguiría un desarrollador desde que recibe una tarea hasta que su código forma parte del proyecto principal. Incluye las etapas de recepción de la tarea, desarrollo de la funcionalidad, pruebas, integración de cambios y actualización del proyecto.



En la práctica, si durante las pruebas la función no se comporta como debe, el desarrollador regresa a corregirla en su propia rama y la vuelve a probar; solo cuando todo funciona se integra al proyecto y se actualiza la versión que comparte el equipo.

Respuestas de análisis

1. ¿Qué ocurriría si dos desarrolladores modifican el mismo archivo simultáneamente?

Con Git no se pierde el trabajo de ninguno de los dos. El sistema intenta combinar ambos cambios automáticamente y, cuando las modificaciones tocan la misma parte del archivo, marca lo que llama un conflicto. Ese conflicto le avisa a los desarrolladores que deben revisar juntos esa sección y decidir cómo dejarla.

2. ¿Por qué es importante conservar un historial de cambios en un proyecto?

Porque ese historial es como una memoria del proyecto. Permite saber qué se hizo, cuándo y por quién, y sobre todo permite regresar a una versión anterior cuando algo se rompe. Si una actualización introduce un error, no hace falta adivinar: se revisa el historial, se identifica el cambio que lo causó y se puede deshacer. También ayuda a entender la evolución del sistema y facilita el trabajo de quien se incorpore más tarde al equipo.

3. ¿Qué beneficios aporta trabajar mediante ramas independientes?

Las ramas independientes permiten que cada desarrollador avance en su tarea sin afectar el trabajo de los demás ni la versión estable. Se puede experimentar o construir una función nueva con tranquilidad, porque mientras esa rama no se una al proyecto principal, nada de lo que ahí se haga puede romper lo que ya funciona. Además, ordena el trabajo: cada rama corresponde a una tarea concreta, lo que hace más fácil revisar y entender los cambios antes de integrarlos.

4. ¿Cómo ayuda el control de versiones cuando aparece un error después de una actualización?

Ayuda muchísimo, porque gracias al historial se puede comparar la versión que funcionaba con la que falló y ubicar exactamente qué cambio provocó el problema. Si es necesario, se revierte ese cambio y el sistema vuelve a su estado estable mientras se corrige con calma. Sin control de versiones, en cambio, habría que buscar el error a ciegas y existiría el riesgo de no poder recuperar la versión que sí servía.

5. ¿Por qué el control de versiones es indispensable en proyectos desarrollados por varios programadores?

Porque es lo que hace posible que muchas personas trabajen sobre el mismo código sin estorbarse. Coordina los cambios, los combina de forma ordenada, guarda quién hizo qué y protege siempre una versión estable a la que se puede volver. Sin él, un equipo termina pisándose el trabajo, perdiendo avances y acumulando errores, justo como le pasó al equipo de SmartLibrary al principio.

Conclusión

Después de analizar el caso de SmartLibrary entendí que el control de versiones no es solo una herramienta técnica, sino una forma de organizar y proteger el trabajo de un equipo. Los problemas que tuvo el equipo no fueron por falta de capacidad, sino por no contar con un método que llevara registro de los cambios y permitiera combinarlos sin perder información. Aprendí que conservar un historial, trabajar con ramas y definir cuándo integrar los cambios marca una diferencia enorme en el orden y la seguridad del proyecto.

Comprendo también por qué Git se ha convertido en un estándar de la industria: resuelve de manera sencilla y confiable problemas que antes costaban mucho tiempo y trabajo, permite colaborar entre muchas personas, guarda todo el historial y deja siempre una versión estable a la que regresar. Es gratuito, funciona en cualquier sistema y se ha integrado con plataformas que casi todos los equipos usan hoy, lo que lo volvió prácticamente indispensable.